
MASAI: Modular Architecture for Software-engineering AI Agents

Daman Arora*, Atharv Sonwane*, Nalin Wadhwa*
Abhav Mehrotra, Saiteja Utpala, Ramakrishna Bairi
Aditya Kanade, Nagarajan Natarajan

Microsoft Research India

{daman1209arora, atharvs.twm, nalin.wadhwa02}@gmail.com

{abhavm1, saitejautpala}@gmail.com

{ram.bairi, kanadeaditya, nagarajan.natarajan}@microsoft.com

Abstract

A common method to solve complex problems in software engineering, is to divide the problem into multiple sub-problems. Inspired by this, we propose a Modular Architecture for Software-engineering AI (MASAI) agents, where different LLM-powered sub-agents are instantiated with well-defined objectives and strategies tuned to achieve those objectives. Our modular architecture offers several advantages: (1) employing and tuning different problem-solving strategies across sub-agents, (2) enabling sub-agents to gather information from different sources scattered throughout a repository, and (3) avoiding unnecessarily long trajectories which inflate costs and add extraneous context. MASAI enabled us to achieve the highest performance (28.33% resolution rate) on the popular and highly challenging SWE-bench Lite dataset consisting of 300 GitHub issues from 11 Python repositories. We conduct a comprehensive evaluation of MASAI relative to other agentic methods and analyze the effects of our design decisions and their contribution to the success of MASAI.

1 Introduction

Software engineering is a challenging activity which requires exercising various skills such as coding, reasoning, testing, and debugging. The ever growing demand for software calls for better support to software engineers. Recent advances in AI offer much promise in this direction.

Large language models (LLMs) have shown remarkable ability to code (Chen et al. [2021], Roziere et al. [2023], CodeGemma Team [2024], *inter alia*), reason [Kojima et al., 2022] and plan [Huang et al., 2022]. Iterative reasoning, structured as chains [Wei et al., 2022] or trees [Yao et al., 2024] of thought, further enhance their ability to solve complex problems that require many inter-related steps of reasoning. When combined with tools or environment actions [Yao et al., 2023, Patil et al., 2023,

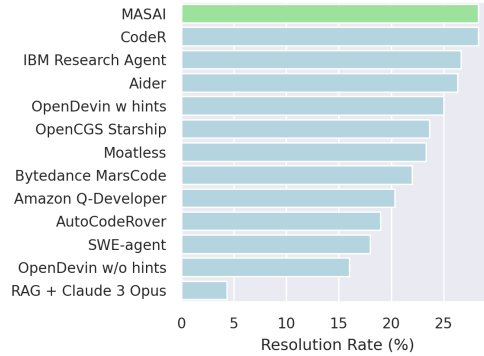


Figure 1: Comparison of MASAI with existing methods. *Resolution rate* refers to the percentage of issues in SWE-bench Lite that are resolved.

* Equal contribution; names listed in alphabetical order

Schick et al., 2024] and feedback from the environment [Zhou et al., 2023, Shinn et al., 2024], they enable autonomous agents capable of achieving specific goals [Zhang et al., 2023].

As the problem complexity increases, it becomes difficult to devise a single, over-arching strategy that works across the board. Indeed, when faced with a complex coding problem, software engineers break it down into sub-problems and use different strategies to deal with them separately. Inspired by this, we propose a Modular Architecture of Software-engineering AI (MASAI) agents, where different LLM-powered sub-agents are instantiated with well-defined objectives and strategies tuned to achieve those objectives.

Our modular architecture consists of 5 different sub-agents: **Test Template Generator** which generates a template test case and instructions on how to run it, **Issue Reproducer** which writes a test case to reproduce the issue, **Edit Localizer** which finds files to be edited, **Fixer** which fixes the issue by generating multiple possible patches, and finally **Ranker** which ranks the patches based on the generated test. When combined, all these individual sub-agents work in tandem to resolve complex real-world software engineering issues.

Our approach offers several advantages: (1) employing and tuning different problem-solving strategies across sub-agents (e.g., ReAct or CoT), (2) enabling sub-agents to gather information from different sources scattered throughout a repository (e.g., from a README or a test file), and (3) avoiding unnecessarily long trajectories which inflate inference costs and pass extraneous context which could degrade performance [Shi et al., 2023].

We evaluate MASAI on the popular and highly challenging SWE-bench Lite dataset [Jimenez et al., 2024] of 300 GitHub issues from 11 Python repositories. Due to its practical relevance and challenging nature, SWE-bench Lite has attracted significant efforts from academia, industry and start-ups. As shown in Figure 1, with the highest resolution rate of 28.33%, MASAI achieves state-of-the-art results on SWE-bench Lite. The field of AI agents, and specifically software-engineering AI agents, is nascent and rapidly evolving. In fact, all the existing methods in Figure 1 have been developed within the past three months. Nevertheless, we do compare against them thoroughly.

AI agents for software engineering would encounter many common sub-problems, such as autonomously understanding testing infrastructure and code organization of a repository, writing new tests, localizing bugs, editing large files without introducing syntactic/semantic errors, synthesizing fixes and writing new code. We believe that it is crucial to understand how different strategies perform on these sub-problems. Therefore we conduct a thorough investigation into the performance of MASAI and existing methods on SWE-bench Lite, and present the impact of key design decisions.

In summary, our contributions are:

1. Propose a modular architecture, MASAI, that allows optimized design of sub-agents separately while combining them to solving larger, end-to-end software engineering tasks.
2. Show the effectiveness of MASAI by achieving the highest resolution rate on SWE-bench Lite.
3. Conduct a thorough investigation into key design decisions of MASAI and the existing methods which can help inform future research and development in this rapidly evolving space.
4. Contribute our results to the SWE-bench Lite leaderboard [MASAI, 2024] for validation.

2 MASAI Agent Architecture

Solving a problem in a code repository requires understanding the problem description and the codebase, gathering the necessary information scattered across multiple files, locating the root cause, fixing it and verifying the fix. Instead of treating this as one long chain of reasoning and actions, we propose modularizing the problem into sub-problems and delegating them to different sub-agents.

2.1 Agent Specification and Composition

A MASAI *agent* is a composition of several MASAI *sub-agents*. A MASAI *sub-agent* is specified by a tuple $\langle \text{Input}, \text{Strategy}, \text{Output} \rangle$ where

1. *Input* to the sub-agent comprises of the code repository, information obtained from other sub-agents as necessary, a set of allowed actions and task instructions.

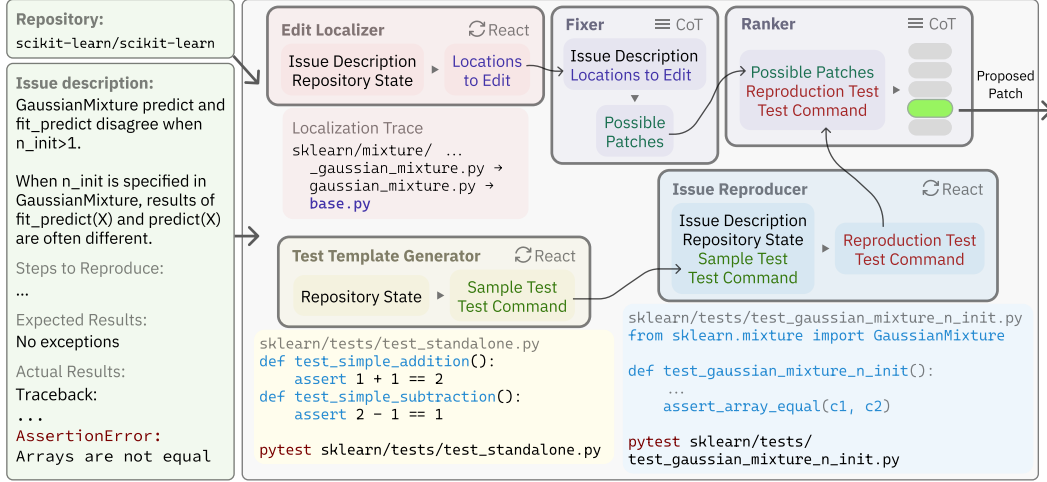


Figure 2: MASAI applied to the task of repository-level issue resolution on an example. MASAI takes a repository and an issue description as input, and produces a single patch. The 5 sub-agents (thick borders) tackle different sub-problems. The information flow between them is shown by directed edges. They marked with the solution strategy and input ▶ output specification

Detail: The example issue from the scikit-learn repository (id: 13142) describes inconsistent behaviour between two functions relating to GaussianMixture and gives an example. The **Test Template Generator** first generates an example test case along with the command to run the test case. This is given to the **Issue Reproducer** which writes a test that reproduces the issue along with the command to run it. The **Edit Localizer** navigates the repository to find files related to the buggy behaviour of GaussianMixture and passes the information to the **Fixer** which generates a set of possible patches which could fix the issue. Finally, the **Ranker** takes in the possible patches and the generated reproductions test. Running the tests on the patches, the ranker observes that one of the patches pass while the rest patch fail. It outputs the ranking with the passing patch first which gets selected as the proposed solution.

2. *Strategy* is the problem-solving strategy to be followed by the sub-agent in using the LLM to solve its given sub-problem. This could be vanilla completion, CoT [Wei et al., 2022], ReAct [Yao et al., 2023], RAG [Lewis et al., 2020], etc.;
3. *Output* is the specification of the content that the sub-agent must return upon completion as well the format it must be presented in.

Compared to multi-agent frameworks [Wu et al., 2023, Qian et al., 2023, Hong et al., 2024], the MASAI architecture is simpler, in that, the sub-agents are given modular objectives that do not require explicit one-to-one or group conversations between sub-agents. The sub-agents are composed by passing the output from one sub-agent to the input of another sub-agent.

2.2 Action Space

All the sub-agents are presented with a set of actions which allows them to interact with the environment. The actions we use in this work are:

1. **READ(file, class, function):** Query and read a specific function, class or file. All three attributes are not necessary; the agent can specify only a function and a file or even a single file. If there exists only one exactly matching code segment with these attributes, then that code is returned. If there are multiple matches, all their names are returned and the query can be refined if necessary. The READ action returns a **lazy representation** that aims to keep the output concise. When reading a file, only signatures of the top level definitions are presented; when reading a class, the signature of the class (class name and member signatures) are presented and when reading a function, its complete body is presented.

2. `EDIT(file, class, function)`: Marks a code segment for editing. Just like `READ`, this marks a code segment only when a unique match exists. Otherwise, the set of partial matches are returned which may be refined further.
3. `ADD(file)`: Marks a file for code addition. The file must exist for the action to succeed.
4. `WRITE(file, contents)`: Writes the specified content to a file. The specified file can be new or a file that the agent has created earlier.
5. `LIST(folder)`: Lists folder contents if it exists.
6. `COMMAND(command)`: Executes the command in a shell with timeout and truncation of large results.
7. `DONE`: Used by the agent to signal that it has completed its assigned objective.

2.3 Agent Instantiation

In this work, we focus on the general task of resolving repository-level issues, as exemplified by the SWE-bench Lite dataset. A problem statement consists of an issue description and a repository. The agent is required to produce a patch so that the issue is resolved. Issue resolution is checked by ensuring that the relevant, held-out test cases pass. Below, we refer to ReAct Yao et al. [2023] which is a problem-solving strategy that alternates between generating an action to take using an LLM followed by executing the action and using the resulting observations as input for the subsequent action generation. Chain of Thought (CoT) Wei et al. [2022] generates solutions to a problem using an LLM while asking it to generate specific intermediate reasoning steps.

We instantiate 5 sub-agents to collectively resolve repository-level issues. Figure 2 shows the overall architecture of our MASAI agent on a concrete example, along with the information flow between the sub-agents (shown by the solid edges).

(1) Test Template Generator: Discovers how to write and run a new test by analyzing the testing setup specific to the repository.

- *Input*: The repository state (within its execution environment) is provided. `READ`, `LIST`, `COMMAND`, `WRITE` and `DONE` actions are provided.
- *Strategy*: ReAct.
- *Output*: The code for a template test case (which is issue independent) for the repository along with the command to run it. This is used to aid the Issue Reproducer sub-agent described next.

Test Template Generator is instructed to explore the documentation and existing tests within the repository to complete its objective and to keep trying until it comes up with a template and a command that passes without exceptions. Test Template Generator evaluates the output of its ReAct loop to determine whether the generated test passes without exceptions. It retries upto a specified limit or until it finds a template that works.

(2) Issue Reproducer: Writes a test that reproduces the behaviour reported in the given issue.

- *Input*: In addition to the repository state and issue description, the sample test file and the command to run it, generated by the Test Template Generator, are provided. Actions available are `READ`, `LIST`, `COMMAND`, `WRITE` and `DONE`.
- *Strategy*: ReAct.
- *Output*: The code for a test case which reproduces the issue and would show a change in status (pass vs. fail) when the issue is fixed. It also outputs the shell command to run the test.

(3) Edit Localizer: Navigates the repository and identifies code locations (files, classes, functions) that need to be edited to resolve the issue.

- *Input*: The repository state and the issue description are provided. Available actions are `READ`, `LIST`, `EDIT`, `ADD`, `COMMAND` and `DONE`.
- *Strategy*: ReAct.
- *Output*: List of code locations (specified through the `EDIT` and `ADD` commands) to edit.

If no locations have been marked at the end of the ReAct loop, then the Edit Localizer selects a set of locations from all of the ones it has read so far.

(4) Fixer: Suggests multiple potential patches to the code locations marked by Edit Localizer that may resolve the issue.

- *Input:* Issue description along with contents of the code locations required to be edited. No actions are given to this sub-agent.
- *Strategy:* CoT.
- *Output:* Multiple possible candidate patches to the provided suspicious code.

When prompting the LLM for a possible patch, Fixer asks for the edit in the form of a **minimal rewrite** instead of rewriting the full sections. Similar to Deligiannis et al. [2023], the content of the locations to edit are provided by Fixer with line numbers. For each edit, the Fixer expects the LLM to output the original version of the code snippet (*pre*) followed by the edited version of this snippet (*post*). Both these snippets are expected to have a line number for each line. Fixer then searches for the *pre* snippet using line numbers in the target file to replace with the *post* version. If an exact match is not found, it uses **fuzzy matching** to find the closest matching span for the *pre* snippet. After replacing with the *post* span, it computes the diff of the target file with its contents before the edit. Syntactically incorrect edits are rejected and the resultant patches are used downstream.

(5) Ranker: Ranks the candidate patches from the Fixer, using the test generated by Issue Reproducer.

- *Input:* Issue description, candidate patches from Fixer, and the reproduction test (as well as the command to run it) from Issue Reproducer. No environment actions are allowed.
- *Strategy:* CoT.
- *Output:* Ranking of the candidate patches in the order of likelihood to resolve the issue.

For each of the patches, Ranker first runs the test on each of the patches and then asks the LLM to determine whether the application of that patch to the repository has caused the provided test to change status (go from failing to passing or vice versa) given the test results. Based on the output of this, the LLM is then asked to rank the patches. The top ranked patch is selected as the issue resolution. If the Issue Reproducer sub-agent could not generate a test, then the Ranker ranks the patches using only the issue description.

3 Experimental Setup

Dataset: We perform experiments on SWE-bench Lite [Jimenez et al., 2024], a collection of 300 software engineering tasks (predominantly bug fixes) sourced from 11 open-source repositories. Each task consists of an issue description and the state of the repository on which the issue was raised. The objective is to produce a patch (that applies to one or more files), which when applied to the repository at the given state, resolves the issue. The proposed patch for an issue is said to successfully resolve, if the targeted suite of tests, provided as part of the dataset (and revealed only at the time of evaluation), passes on the patched version of the repository. Each task consists of an issue description and the state of the repository on which the issue was raised. The objective is to produce a patch given a repository and an issue description, so that the repository after the patch is applied, passes the issue-specific tests (that are never revealed to the agent).

Metrics: We report three metrics: (1) *Resolution rate*, the percentage of issues successfully resolved (i.e., pass the issue-specific tests); (2) *Localization rate*, the percentage of issues where the patch proposed by a method fully covers the ground-truth patch files, i.e., where recall is 100% at the file level; and (3) *Application rate*, the percentage of issues where the patch proposed by a method successfully applies on the repository (i.e., the Linux command `patch` does not raise an error).

Competing methods: We compare with all the existing methods that are also evaluated on SWE-bench Lite (with logs here):

1. **SWE-agent** [Yang et al., 2024a]: Utilizes a single ReAct loop along with specialized environment interface with multiple tools. Uses GPT-4 (1106).

2. **AutoCodeRover** [Zhang et al., 2024] (ACR): Uses a ReAct loop for localization and another for generating patches. Uses specialized tools for searching specific code elements (class, method) within other code elements and presenting them as signatures whenever appropriate. Uses GPT-4 (0125).
3. **OpenDevin** [OpenDevin, 2024]: Uses the CodeAct [Wang et al., 2024a] framework where the agent (a single ReAct loop) can execute any bash command along with using various helper commands. The version of OpenDevin with highest reported performance `v1.3_gpt4o` makes use of `hints_text` in SWE-bench Lite, conversation transcript of developers on an issue in GitHub. While we include results from this version, we compare in detail with the highest performing version that does *not* use hints, `v1.5_gpt4o_nohints`.
4. **Aider** [Aider, 2024]: Uses static analysis to provide a compact view of the repository and, in turn, to determine the file(s) to edit. Limited number of ReAct steps are taken to make an edit to the identified file(s) and iteratively update it until it is syntactically correct and passes existing tests. After these steps, the final status of linting and pre-existing tests are used to determine whether the tool should be run again from scratch until a plausible solution is found. Uses GPT-4o and Claude 3 Opus on alternate runs
5. **CodeR** [Chen et al., 2024]: A multi agent solution with separate agents to reproduce the issue, localize the fault and iteratively edit the code to resolve the issue. Uses BM25 along with test coverage statistics for fault localization. Uses GPT-4 (1106).
6. **Moatless** [Moatless Tools, 2024]: Uses a ReAct loop to localize and another to fix the code. Leverages a semantic search tool that searches with natural language queries for relevant code chunks in the repository.
7. **RAG**: Uses BM25 to retrieve relevant files which are used to prompt an LLM to generate a patch. We compare with the best-performing RAG model from the SWE-bench Lite leaderboard [SWE-bench, 2024]: RAG + Claude 3 Opus.
8. Along with the above, commercial offerings **Amazon Q-Developer** [Amazon, 2024], **Bytedance MarsCode** [Bytedance, 2024], **OpenCGS Starship** [OpenCGS, 2024] and **IBM Research Agent-101** [IBM, 2024] have also reported results on SWE-bench Lite. While we report metrics for these, we are unable to conduct further comparisons with them due to non-availability of detailed logs or any information about their approaches. We do not compare with Devin [Devin, 2024] as it reports performance a subset of SWE-bench different from SWE-bench Lite.

Implementation: We evaluate MASAI by setting up a fresh development environment with all the requirements and providing the issue description. MASAI generates a single patch which is then evaluated using the SWE-bench Lite testing harness. The `tree-sitter==0.21.1` package is used to implement the lazy representation part of the READ function. We use the GPT-4o model throughout our pipeline. For Test Template Generator, we start with a temperature of 0 and increase by 0.2 for each attempt. For Issue Reproducer, Edit Localizer, and Ranker, we use a temperature of 0; for Fixer, we use 0.5 and sample 5 candidate patches. We limit the ReAct loops of the Test Template Generator, Issue Reproducer, and Edit Localizer to 25 steps and limit Test Template Generator to 3 retries. After the ranker selects the patch, we run an auto-import tool to add missing imports. We discard any edits to pre-existing tests which the agent might have made. The per-issue cost for MASAI is 1.96 USD on average. We estimate the total cost of our experiments to be <10k USD.

4 Results

We first present comprehensive results on the SWE-bench Lite dataset. Then we provide supporting empirical observations and examples that bring out the effectiveness of our design choices.

4.1 RQ1: Performance on software engineering tasks in SWE-bench Lite

We present our main results in Table 1. Multiple remarks are in order.

1. Our method, MASAI, achieves the highest resolution rate of 28.33% on the dataset, thereby establishing a state-of-the-art on the benchmark leaderboard alongside CodeR [MASAI, 2024].

Method	Resolution rate (%)	Localisation rate (%)	Application rate (%)
RAG	4.33	48.00	51.67
SWE-agent	18.00	61.00	93.67
ACR	19.00	62.33	80.00
Q-Dev	20.33	71.67	97.33
MarsCode	22.00	67.00	83.67
Moatless	23.33	73.00	97.00
Starship	23.67	90.67	99.00
OpenDevin	25.00	77.00	90.00
– hints	16.00	63.00	81.33
Aider	26.33	69.67	96.67
Agent-101	26.67	72.67	97.33
CodeR	28.33	66.67	74.00
MASAI	28.33	75.00	95.33

Table 1: Performance of baseline and competing methods on SWE-bench Lite (best in **bold**). Our proposed method, MASAI, achieves the best resolution rate (% issues resolved). Row “– hints” indicates executing OpenDevin without using `hints_text` in the dataset.

2. Standard RAG baseline (first row) performs substantially poor on the dataset, as has also been established in recent works [Jimenez et al., 2024, Chen et al., 2024]; which is a strong indication of the complexity of the SWE-bench Lite dataset.
3. MASAI localizes the issue (at a file-level) in 75% of the cases; the best method in terms of localization rate, OpenCGS Starship, at nearly 91%, however achieves only 23.67% resolution rate.
4. The (edit) application rate is generally high for all LLM-based agents; MASAI’s patches, in particular, successfully apply in over 95% of the cases.

4.2 RQ2: Assumptions by different methods

High autonomy and less dependence on external signals (e.g., expert hints) is desirable from software-engineering agents. In the standard SWE-bench Lite setup, all agents are provided the issue description along with the repository. However, we observe that different methods make different assumptions about available auxiliary information.

- All methods apart from RAG and Moatless require that for each task, an environment be set up with the appropriate requirements installed beforehand so that code can be executed.
- OpenDevin avails `hints_text` provided by SWE-bench Lite as discussed in Section 3.
- Aider, when running pre-existing tests, uses pre-determined test commands consist of (1) the testing framework used to run tests in the task repository and (2) specific unit tests that target the code pertaining to the issue at hand. The former assumes information about the repository-specific testing framework which is not present in the standard SWE-bench Lite setup. In the case of the latter, providing output from only the target test (and not the whole test suite) during ReAct steps, inadvertently provides additional information about which part of the repository is relevant to the issue.
- CodeR uses coverage-based code ranking [Wong et al., 2016] for fault localization. As in Aider, this would require repository-specific commands to run pre-existing tests, and instrumentation of the full repository to get coverage information. However from the available trajectory logs of CodeR it does not appear to discover these autonomously.

MASAI aims for high autonomy by avoiding dependence on additional inputs, only relying on the original setup proposed by Jimenez et al. [2024]. SWE-agent and AutoCodeRover operate at a similar level of autonomy to MASAI. Results in Table 1 show that MASAI outperforms all other approaches without making additional assumptions.

Selection Strategy	1 Sample	5 Samples
Oracle	23.33%	35.00%
Random	-	22.28%
LLM w/o test	-	23.33%
LLM w/ test (Ranker)	-	28.33%

Table 2: Resolution rates of MASAI on SWE-bench Lite, with different number of Fixer samples (i.e., candidate patches), using different sample selection strategies (rows, discussed in Section 4.4).

4.3 RQ3: How does MASAI perform effective fault localization from issue description?

Localization requires multi-step reasoning to identify the root cause of the error from issue descriptions which are often vague and usually only describe the problem being observed. We observe that (1) the choice of ReAct as the strategy, (2) the specificity of its objective (to only identify files to edit) and (3) the designs of tools available enables the Edit Localizer to perform the required multi-step reasoning in a flexible and robust manner. Note that (1) and (2) are results of the modularity of MASAI. SWE-agent and OpenDevin, methods that do not employ a separate localization sub-agent, achieve 61% and 63% localization rates respectively, compared to 75% achieved by MASAI’s Edit Localizer.

We observe the advantages of using a ReAct sub-agent, by comparing with Aider which uses a single step CoT approach. In the 27 issues solved by MASAI but not by Aider, Aider failed to localize in 10 (37%) issues whereas among the 21 issues solved by Aider but not by MASAI, MASAI only failed to localize in 3 (14%) issues. This shows that better localization plays a role in superior resolution rate. Comparing the average search steps (as proxy for complexity) required for problems that both Aider and MASAI solved (10.9) and those that only MASAI solved (12.8), we further see that MASAI’s ReAct based Edit Localizer has the flexibility to scale to more complex localization challenges.

[Example 1]: MASAI performs **multi-step reasoning** required for localization in the task `scikit-learn__scikit-learn-13142` (described in Fig. 2). Edit Localizer finds the class mentioned in the issue and then traces symbols and inheritance links to identify the root cause.

[Example 2]: The ability of the READ action to return **approximate matches** (Section 2) helps in the issue `astropy__astropy-14995`. When the LLM asks for a non-existent `NDDataRef.multiply` method in the `astropy/nddata/nddata.py` file, the action responds with an approximate match `NDArithmeticMixin.multiply` in a different file `astropy/nddata/mixins/ndarithmetric.py`. Then the sub-agent traces 3 callee links to get to the actual faulty function.

[Example 3]: Access to basic **shell commands** helps the Edit Localizer in the issue `matplotlib__matplotlib-25332`. `grep` is used to look for occurrences of the `FigureBase._align_label_groups` attribute within the large file `lib/matplotlib/figure.py`. From the occurrences (output from `grep`), MASAI finds out that the attribute is set using `cbook.Grouper()` – the class that needs to be edited to resolve the issue.

Neither Aider nor CodeR localized faulty functions correctly in any of the 3 examples. OpenDevin localized Example 2; SWE-agent Examples 2 and 3.

4.4 RQ4: How does MASAI’s sampling and ranking compare to iterative repair?

We observe that sampling multiple repair patches from the Fixer significantly increases the possibility of generating a correct patch, as reported in Table 2 (Oracle selection 23.33% at 1 sample vs 35% at 5 samples). However the LLM alone is unable to select amongst these patches (LLM w/o test). This can be overcome by using the output from the generated issue-reproduction test on each patch for ranking the patches (LLM w/ test (Ranker)).

MASAI exploits the above observations through its modularity by (1) leveraging a CoT sampling strategy for Fixer and (2) instantiating independent sub-agents for test generation and repair. Other methods rely on an iterative approach to extract multiple edits from the LLM asking it to iteratively fix any mistakes it has made.

Method	Both localised	Method resolved	MASAI resolved
RAG	126	12	52 (+ 31.7%)
ACR	166	51	73 (+ 13.2%)
Q-Dev	191	55	75 (+ 10.5%)
SWE-agent	166	48	65 (+ 10.2%)
Starship	220	62	81 (+ 8.6%)
OpenDevin	187	60	74 (+ 7.5%)
– hints	164	39	67 (+ 17.1%)
Moatless	193	62	75 (+ 6.7%)
MarsCode	182	59	71 (+ 6.6%)
Agent-101	193	69	72 (+ 1.6%)
Aider	189	71	71 (=)
CodeR	174	77	72 (- 0.3%)

Table 3: Number of issues resolved by a method (Method resolved) named in the rows and by MASAI (MASAI resolved) among the issues that are successfully localized by both MASAI and the method (“Both localised” column, out of 300). Row-wise max. in bold.

We evaluate the benefits of our approach empirically in Table 3. By controlling for localization, we are comparing the effectiveness of completing the repair. MASAI is substantially more effective at this than most methods, barring CodeR and Aider.

As an example, consider the issue `django__django-14787` where CodeR, Aider, OpenDevin and MASAI all correctly localize the issue, but only MASAI solves it correctly. While iterative methods sample one candidate and keep refining it without success, MASAI’s Fixer sub-agent generates 5 samples out of which only one is correct – demonstrating the importance for diverse sampling. MASAI’s Ranker correctly ranks these by utilizing outputs from running the generated reproduction test. Aider submits patch which passes pre-existing tests but is actually incorrect, showing the importance of the generated reproduction test to eliminate false positives.

4.5 RQ5: How does MASAI perform effective issue reproduction?

As discussed in the previous RQ, the ability to generate tests that reproduce the stated issue is critical to select Fixer samples. Often repositories employ uncommon testing frameworks, that makes this task hard. Consider the issue `django__django-14672`. This repository proved hard to write tests for since it uses a custom testing framework, which involved having all new test classes derive from a certain base class to run. OpenDevin was unable to reproduce the test; in its attempt to install pytest, it ran out of budget and failed to solve this issue.

To remedy this, we decompose test reproduction into two steps: (1) Test Template Generator reads documentation/existing tests to **generate a sample test template** and instructions to run; (2) Issue Reproducer then uses the template as an example to **create an issue specific test**. This improves the overall capability of reproducing tests in MASAI, Test Template Generator first goes through the repository, creates a template file that correctly makes use of `django.test.TestCase` to create an example test case as well as the correct command to run it. The Issue Reproducer subsequently reproduces the issue correctly, without running into problems that OpenDevin faced.

4.6 RQ6: How does MASAI generate edits that can be applied successfully?

The representation used to encode edits can have a large impact on the performance. As discussed in Section 2, MASAI prompts the LLM for edits, in the form of a **minimal rewrite** — to reproduce the current state of the code snippet it wants to edit, followed by the edited version of this snippet. Recall that we also employ **fuzzy matching** to find the relevant span in the file, by searching for the snippet that best fuzzily matches with the one provided by the model. This mitigates copying or line counting mistakes by the LLM, significantly reducing the number of syntax errors introduced when editing. Our edit representation and fuzzing matching together yield 96.33% edit application rate (Table 1) which is among the highest.

5 Related Work

We have already discussed competing methods evaluated on SWE-bench Lite, in Sections 3 and 4. We now highlight other related work on LLM-powered agents.

Software-engineering agents: Language Agent Tree Search Zhou et al. [2023] synergizes reasoning, planning, and acting abilities of LLMs. Their strategy relies on determining partial or full termination of the search (e.g., by running provided golden test cases for successful code generation as in HumanEval Chen et al. [2021]) and backtracking if necessary; this is often infeasible in complex software engineering tasks we tackle in this paper. CodePlan [Bairi et al., 2023] combines LLMs with static analysis-backed planning for repository-level software engineering tasks such as package migration. It relies on compiler feedback and dependency graphs to guide the localization of edits; unlike in our general setting, where the agents are more autonomous, and are equipped to discover localization strategies. AlphaCodium [Ridnik et al., 2024] differs from MASAI in that (1) it uses public *and* AI-generated test cases for filtering; (2) is evaluated in the generation (NL2Code) setting.

Conversational and multi-agent frameworks: In this line of work Guo et al. [2024], Yang et al. [2024b], (1) the focus is often on the high level aspects of agent design such as conversation protocols. AutoGen [Wu et al., 2023] and AgentVerse [Chen et al., 2023] provide abstractions for agent interactions and conversational programming for design of multi-agent systems; similarly, Dynamic agent networks [Liu et al., 2023] focuses on inference-time agent selection and agent team optimization; and (2) the frameworks are typically instantiated on standard RL or relatively simpler code generation datasets. For instance, AutoDev [Tufano et al., 2024] can execute actions like file editing, retrieval, testing, but is evaluated on the HumanEval [Chen et al., 2021] NL2Code dataset. Similarly, MetaGPT [Hong et al., 2024] and ChatDev [Qian et al., 2023], dialogue-based cooperative agent frameworks, are instantiated on generation tasks involving a few hundred lines of code.

In contrast, we focus on designing a modular agent architecture for solving complex, real-world software engineering tasks, as exemplified by the SWE-bench Lite dataset.

Divide-and-Conquer approaches: In this line of work, the given complex task is broken down into multiple sub-goals that are solved individually, and then the solution for the task is synthesized. Multi-level Compositional Reasoning (MCR) Agent [Bhambri et al., 2023] uses compositional reasoning for instruction following in environments with partial observability and requiring long-horizon planning, such as in robotic navigation. Compositional T2I [Wang et al., 2024b] agent uses divide-and-conquer strategy for generating images from complex textual descriptions. SwiftSage [Lin et al., 2024] agent, inspired by the dual-process theory of human cognition for solving tasks, e.g., closed-world scientific experiments [Wang et al., 2022], uses finetuned SLM policy (“Swift”) to decide and execute fast actions, and an LLM (“Sage”) for deliberate planning of sub-goals and for backtracking when necessary.

6 Conclusions

As divide-and-conquer helps humans overcome complexity, similar approaches to modularize tasks into sub-tasks can help AI agents as well. In this work, we presented a modular architecture, MASAI, for software-engineering agents. Encouraged by the effectiveness of MASAI on SWE-bench Lite, we plan to extend it to a larger range of software-engineering tasks, which will also involve building realistic and diverse datasets.

7 Limitations

Our evaluation is centered on the widely-used SWE-bench Lite dataset for evaluating software-engineering AI agents. It allowed us to do head-to-head comparison with many agents. However, the breadth of issues covered in SWE-bench Lite is limited to those that can be validated using tests. In future, we expect us and the community to expand the scope to more diverse issues.

There are a number of LLMs that support code understanding and generation. The modularity of MASAI permits use of different language models in different sub-agents. Due to the time and cost

constraints, we have instantiated all sub-agents with GPT-4o. The cost-performance trade-off of using different LLMs and possibly, even small language models (SLMs) is an interesting research problem. The competing methods that we compared against do employ different LLMs, but this still leaves out direct comparison of different LLMs on a fixed solution strategy.

The issue descriptions in SWE-bench Lite are all in English. This leaves out issues from a large segment of non-English speaking developers. The increasing support for the diverse world languages by LLMs should enable multi-lingual evaluation even in the software engineering domain, which is a problem that we are excited about.

8 Broader Concerns

Agentic frameworks with the ability to use tools like shell commands can lead to unintended side-effects on the user’s system. Appropriate guardrails and sandboxing can mitigate such problems.

Our approach contributes towards the development of tools to autonomously perform software development tasks. This raises various security concerns. The tool may not always follow best practices when writing or editing code, leading to introduction of security vulnerabilities and bugs. Therefore, it is important for code changes suggested by the tool to be reviewed by expert developers before being deployed to real world systems.

As mentioned in the Section 7, the dataset we evaluate on (SWE-bench Lite) as well as the model we use (GPT-4o) are primarily in English. This limits the usability of our tool to software engineers proficient in English. Further work is necessary in developing methods for non-English speaking developers in order to prevent this population from being marginalized.

References

- Aider. <https://aider.chat/2024/06/02/main-swe-bench.html>, 2024.
- Amazon. <https://aws.amazon.com/q/developer/>, 2024.
- Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, Arun Iyer, Suresh Parthasarathy, Sriram Rajamani, B Ashok, Shashank Shet, et al. CodePlan: Repository-level coding using LLMs and planning. *arXiv preprint arXiv:2309.12499*, 2023.
- Suvaansh Bhambri, Byeonghwi Kim, and Jonghyun Choi. Multi-level compositional reasoning for interactive instruction following. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 223–231, 2023.
- Bytedance. <https://www.marscode.com/>, 2024.
- Dong Chen, Shaoxin Lin, Muhan Zeng, Daoguang Zan, Jian-Gang Wang, Anton Cheshkov, Jun Sun, Hao Yu, Guoliang Dong, Artem Aliev, et al. CodeR: Issue resolving with multi-agent and task graphs. *arXiv preprint arXiv:2406.01304*, 2024.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021.
- Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chen Qian, Chi-Min Chan, Yujia Qin, Yaxi Lu, Ruobing Xie, et al. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors in agents. *arXiv preprint arXiv:2308.10848*, 2023.

- CodeGemma Team. CodeGemma: Open Code Models Based on Gemma. 2024.
- Pantazis Deligiannis, Akash Lal, Nikita Mehrotra, and Aseem Rastogi. Fixing rust compilation errors using llms. *arXiv preprint arXiv:2308.05177*, 2023.
- Devin. Introducing Devin, the first AI software engineer. <https://www.cognition.ai/blog/introducing-devin>, 2024.
- Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V Chawla, Olaf Wiest, and Xiangliang Zhang. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680*, 2024.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. MetaGPT: Meta programming for Multi-Agent Collaborative Framework. In *The Twelfth International Conference on Learning Representations*, 2024.
- Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *International Conference on Machine Learning*, pages 9118–9147. PMLR, 2022.
- IBM. https://github.com/swe-bench/experiments/tree/main/evaluation/lite/20240612_IBM_Research_Agent101, 2024.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can Language Models Resolve Real-world Github Issues? In *The Twelfth International Conference on Learning Representations*, 2024.
- Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35: 22199–22213, 2022.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33: 9459–9474, 2020.
- Bill Yuchen Lin, Yicheng Fu, Karina Yang, Faeze Brahman, Shiyu Huang, Chandra Bhagavatula, Prithviraj Ammanabrolu, Yejin Choi, and Xiang Ren. Swiftsage: A generative agent with fast and slow thinking for complex interactive tasks. *Advances in Neural Information Processing Systems*, 36, 2024.
- Zijun Liu, Yanzhe Zhang, Peng Li, Yang Liu, and Diyi Yang. Dynamic LLM-agent network: An LLM-agent collaboration framework with agent team optimization. *arXiv preprint arXiv:2310.02170*, 2023.
- MASAI. <https://github.com/swe-bench/experiments/pull/20>, 2024.
- Moatless Tools. <https://github.com/aorwall/moatless-tools>, 2024.
- OpenCGS. <https://opencsg.com/product?class=StarShip>, 2024.
- OpenDevin. <https://opendevin.github.io/OpenDevin/>, 2024.
- Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis. *arXiv preprint arXiv:2305.15334*, 2023.
- Chen Qian, Xin Cong, Wei Liu, Cheng Yang, Weize Chen, Yusheng Su, Yufan Dang, Jiahao Li, Juyuan Xu, Dahai Li, et al. Communicative agents for software development. *arXiv preprint arXiv:2307.07924*, 2023.
- Tal Ridnik, Dedy Kredo, and Itamar Friedman. Code generation with alphacodium: From prompt engineering to flow engineering. *arXiv preprint arXiv:2401.08500*, 2024.

- Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 2024.
- Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H. Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *ICML*, volume 202 of *Proceedings of Machine Learning Research*, pages 31210–31227. PMLR, 2023.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2024.
- SWE-bench. <https://www.swebench.com/>, 2024.
- Michele Tufano, Anisha Agarwal, Jinu Jang, Roshanak Zilouchian Moghaddam, and Neel Sundaresan. AutoDev: Automated AI-Driven Development. *arXiv preprint arXiv:2403.08299*, 2024.
- Ruoyao Wang, Peter Jansen, Marc-Alexandre Côté, and Prithviraj Ammanabrolu. Scienceworld: Is your agent smarter than a 5th grader?, 2022.
- Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. *arXiv preprint arXiv:2402.01030*, 2024a.
- Zhenyu Wang, Enze Xie, Aoxue Li, Zhongdao Wang, Xihui Liu, and Zhenguo Li. Divide and conquer: Language models can plan and self-correct for compositional text-to-image generation. *arXiv e-prints*, pages arXiv–2401, 2024b.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- W Eric Wong, Ruizhi Gao, Yihao Li, Rui Abreu, and Franz Wotawa. A survey on software fault localization. *IEEE Transactions on Software Engineering*, 42(8):707–740, 2016.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. Autogen: Enabling next-gen LLM applications via multi-agent conversation framework. *arXiv preprint arXiv:2308.08155*, 2023.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent computer interfaces enable software engineering language models, 2024a.
- Ke Yang, Jiateng Liu, John Wu, Chaoqi Yang, Yi R Fung, Sha Li, Zixuan Huang, Xu Cao, Xingyao Wang, Yiquan Wang, et al. If LLM is the wizard, then code is the wand: A survey on how code empowers large language models to serve as intelligent agents. *arXiv e-prints*, pages arXiv–2401, 2024b.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36, 2024.
- Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. *arXiv preprint arXiv:2404.05427*, 2024.

Zhuosheng Zhang, Yao Yao, Aston Zhang, Xiangru Tang, Xinbei Ma, Zhiwei He, Yiming Wang, Mark Gerstein, Rui Wang, Gongshen Liu, et al. Igniting language intelligence: The hitchhiker’s guide from chain-of-thought reasoning to language agents. *arXiv preprint arXiv:2311.11797*, 2023.

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning acting and planning in language models. *arXiv preprint arXiv:2310.04406*, 2023.